

Student 38 **SHEKAR DHARANISH** 192472152RD

5 / 9 submitted

Q1. Banking Transaction System – Exception Handling

CODE

```
import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            double balance = Double.parseDouble(sc.next());
            double amount = Double.parseDouble(sc.next());
            if(amount<=0) {
                System.out.println("Invalid withdrawal amount: must be positive.");
            } else if (amount > balance) {
                throw new InsufficientBalanceException("Insufficent balance for this withdrwal.");
            } else {
                balance -= amount;
                System.out.println("Withdrawal successful.remaining balance: "+balance);
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid input:please enter numeric value only.");
        } catch(InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("Transaction completed.");
        }
    }
}
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) { super(msg);}
}
```

INPUT

10000
3000

OUTPUT

Withdrawal successful.remaining balance: 7000.0
Transaction completed.

Q2. Hospital Patient Registration – User-Defined Exception

CODE

```
class Main{
    public static void main(String[] args){
        int age=35;
        double temp=98.6;
        try{
            if(age<18)throw new InvalidAgeException();
            if(temp<95||temp>100)throw new InvalidTemperatureException();
            System.out.println("patient registered sucessfully");
        }
        catch(Exception e){
            System.out.println(e.getMessage());
        }finally{
            System.out.println("Registered completed");
        }
    }
}

class InvalidAgeException extends Exception{
    InvalidAgeException(){
        super("InvalidAgeException");
    }
}

class InvalidTemperatureException extends Exception{
    InvalidTemperatureException(){
        super("InvalidTemperatureException");
    }
}
```

OUTPUT

```
patient registered sucessfully
Registered completed
```

Q3. E-Commerce Order Processing – Built-in Exceptions

CODE

```
class Main{
    public static void main(String[] args){
        int index=1,qty=2;
        double[] price={100,200,300};
        try{
            if(qty<=0)throw new IllegalArgumentException("Invalid quality");
            System.out.println("Total cost:"+price[index]*qty);
        }catch(ArrayIndexOutOfBoundsException e){
            System.out.println("Inavild product index");
        }catch(IllegalArgumentException e)
        {
            System.out.println(e.getMessage());
        }
    }
}
```

OUTPUT

```
Total cost:400.0
```

Q4. Railway Ticket Booking – Multithreading and Synchronization

— Not Submitted —

Q5. Bank Account – Concurrent Withdrawals

— Not Submitted —

Q6. Online Food Delivery – Multiple Threads and Execution Order

CODE

```
class Main{
    static String b;
    static synchronized void Produce(String s)throws Exception{
        while(b!=null)Main.class.wait();
        b=s;
        System.out.println("Produced"+s);
        Main.class.notify();
    }
    static synchronized void consume()throws Exception{
        while(b==null)Main.class.wait();
        System.out.print("Cosumed"+b);
        b=null;
        Main.class.notify();
    }
    public static void main(String[] args)throws Exception{
        Thread t1=new Thread()->{
            try{Produce("p1");
                Produce("p2");
                Produce("p3");}
            catch(Exception e){}
        };
        Thread t2=new Thread()->{
            try{consume();consume();
                consume();}
            catch(Exception e){}
        };
        t1.start();
        t2.start();
        t1.join();
        t2.join();
    }
}
```

OUTPUT

```
Producedp1
Cosumedp1Producedp2
Cosumedp2Producedp3
Cosumedp3
```

Q7. Hospital Emergency System – Thread Priority

CODE

```
class Main {
    static void order(){
        System.out.println("order Processing");
    }
    static void payment() {
        System.out.println("Payment Processing");
    }
    static void delivery(){
        System.out.println("Delivery Assignment");
    }
    public static void main(String[] args)throws Exception {
        Thread t1=new Thread(()->order());
        Thread t2=new Thread(()->payment());
        Thread t3=new Thread(()->delivery());
        t1.start();
        t2.join();
        t2.start();
        t2.join();
        t3.start();
        t3.join();
    }
}
```

OUTPUT

```
order Processing
Payment Processing
Delivery Assignment
```

Q8. Manufacturing Plant – Producer-Consumer Problem

— Not Submitted —

Q9. Online Banking – Exception Handling with Multiple Threads

— Not Submitted —